



An Incremental Bangladesh Sectoral Data Integration Model

Database Systems Project

Scrum Level-1

Scrum Group-3

Report submitted August 2026

A project submitted to **Professor Dr. Rudra Pratap Deb Nath**, Department of Computer Science and Engineering, Chittagong University (CU), in partial fulfillment of the requirements for the Database Systems Lab course. The project is not submitted to any other organization simultaneously. The course teacher holds the copyright. The project cannot be distributed or commercialized without their formal permission.

Group Details

Sl.	Name	Student ID	Session	Signature
1	Md. Alamin Molla	24701076	2023–2024	
2	Sheikh Showkotul Islam Shakib	24701041	2023–2024	
3	Ratul Hasan Nipu	23701060	2022–2023	
4	Jahid Alam Ridoy	23701039	2022–2023	

Table 1: Details of the Project Group

Supervisor: Professor Dr. Rudra Pratap Deb Nath

Supervisor Approval: _____

Date: _____

Preface

This report documents the work carried out at the Department of Computer Science and Engineering, University of Chittagong, during August 2026. The title of the project is *Health Data Integration, Database Design and Entity Relationship Diagram Development using Government Health Reports of Bangladesh*, undertaken as part of the Database Systems Lab course.

This project is part of a larger collaborative effort in which all student groups are working on the same overarching system: **MODELBD**, an integrated database system covering seven distinct data sectors, including Agriculture, Healthcare, Energy, and others. The Scrum of Scrum (3-level) approach is adopted to develop MODELBD. At the bottom-most layer, each group is assigned one sector. Our group is assigned the **Healthcare** sector.

In short, the problem we address is that Bangladesh's health-related information is scattered across many disconnected government documents; our response is to connect that scattered information into a single, structured database. Our work involves collecting government health reports of Bangladesh, processing and extracting structured data from Portable Document Format documents, designing a normalized relational database, and developing conceptual as well as physical Entity Relationship Diagrams.

Md. Alamin Molla

Sheikh Showkotul Islam Shakib

Ratul Hasan Nipu

Jahid Alam Ridoy

Contents

Preface	2
Abstract	7
1 Introduction	9
1.1 Background and Motivation	9
1.2 Problem Statement	10
1.3 System Definition	10
1.4 System Development Process	10
1.5 Organization	11
2 Project Structure and Management	12
2.1 Resource Management	12
2.2 Team Roles and Responsibilities	12
2.3 Tools Used	13
2.4 Scrum Structure	14
2.5 Product Backlog Summary	14
3 Sprint 1	16
3.1 Sprint Backlog and Overview	16
3.2 Week 1: Data Collection and Source Identification	16
3.2.1 Challenges Faced	17
3.2.2 Approach	17
3.2.3 Outcome	17
3.3 Week 2: Automated Portable Document Format Collection and Repository Creation	17
3.3.1 Challenges Faced	17
3.3.2 Tools Used	18
3.3.3 Performance	18
3.3.4 Outcome	18
3.4 Week 3: Portable Document Format Processing and Table Extraction .	18
3.4.1 Challenges Faced	18
3.4.2 Tools Used	19

3.4.3	Performance	19
3.4.4	Outcome	19
3.5	Week 4: Data Cleaning, Transformation and Categorization	19
3.5.1	Data Quality Issues Identified	19
3.5.2	Tools Used	19
3.5.3	Outcome	19
3.6	Week 5: Database Design and Normalization	20
3.6.1	Normalization	20
3.6.2	Worked Example: Normalizing the Patient Table (Unnormalized Form to Boyce–Codd Normal Form)	20
3.6.3	Outcome	25
3.7	Week 6: MySQL Implementation, Validation and Entity Relationship Diagram Development	25
3.7.1	Sample Normalized Tables (1NF to 3NF)	27
3.8	Logical Entity Relationship Diagram Development	30
3.9	Entity-Relationship (ER) Diagram Description	33
3.9.1	Entities and Attribute Details	33
3.9.2	Medical Conditions and Disease Tracking Entities	34
3.9.3	Relationships Between Entities	34
3.10	Data Validation	34
3.11	Final Sprint 1 Deliverables	34
4	Deployment and Maintenance	36
4.1	Repository Structure	36
4.2	How to Run the Pipeline	36
4.3	Maintenance	37
5	Collaboration	38
5.1	Intra-Group Collaboration	38
5.2	Inter-Group Collaboration	38
5.3	Good Memory	39
5.4	Bad Memory	39
6	Comments on Course Pedagogy	40
6.1	Lessons Learned	40
6.2	Suggestions for Future Batches	41
7	Conclusion and Future Work	43
7.1	Conclusion	43
7.2	Limitations	43

7.3	Future Work	43
8	Bibliography	45

List of Tables

1	Details of the Project Group	1
2.1	Team Roles and Responsibilities	13
2.2	Tools and Technologies	14
3.1	Sprint 1 Task Summary	16
3.2	Functional Dependencies Used in the Patient Normalization Example .	21
3.5	Second Normal Form – Table 1	22
3.6	Second Normal Form – Table 2	22
3.7	Third Normal Form – Patient	23
3.8	Third Normal Form – Bed	23
3.9	Third Normal Form – Facility	23
3.10	Third Normal Form – MaternalHealth	23
3.11	Boyce–Codd Normal Form Verification Across All Four Tables	24
3.12	Final Boyce–Codd Normal Form Schema for the Patient Care Record .	24
3.13	21 Normalized MySQL Database Tables	26
3.14	HealthFacility – Sample Normalized Data	27
3.15	HospitalBed – Sample Normalized Data	28
3.16	Designation – Sample Normalized Data	28
3.17	Disease – Sample Normalized Data	29
3.18	HealthWorker – Sample Normalized Data	30
3.19	Sample Entities from Conceptual Entity Relationship Diagram	31
6.1	Course Pedagogy: Opportunities and Limitations	40

Abstract

Motivation

Bangladesh generates a vast amount of health-related data through government ministries, hospital registries, training programs, and annual health bulletins. However, this data remains scattered across hundreds of unstructured PDF documents, making it inaccessible for real-time analysis, planning, and policy-making. This project is motivated by the urgent need to transform these fragmented health datasets into a structured, searchable, and central system. Specifically, the development of an automated disease surveillance and resource forecasting dashboard drives this effort. By digitizing regional disease incidence rates alongside hospital capacity metrics, such as bed availability, ICU usage, and medical supplies, the system provides interactive visualizations and real-time query capabilities. This allows policymakers and healthcare administrators to monitor localized outbreaks instantly, optimize resource distribution, and make data-driven clinical and operational decisions.

Problem Definition: The core problem addressed in this project is the unavailability of machine-readable, structured health data from Bangladesh government sources. Key entities such as hospitals, diseases, health programs, budgets, training activities, cancer registries, and health workers are documented in reports but are not available in a queryable database format.

Limitations of Current Solutions: Currently, health stakeholders must manually browse through numerous Portable Document Format reports to find relevant information. There is no centralized database that integrates data across entities such as `Patient`, `Hospital`, `Disease`, `HealthProgram`, `Budget`, `Training`, and `CancerRegistry`. Inconsistencies in naming conventions and reporting formats across documents further hinder data reuse. This directly connects back to the problem stated above: fragmented documents cannot, on their own, answer cross-cutting questions that stakeholders actually need answered.

Our Approach: We adopt an incremental, sprint-based development process using Scrum methodology. The approach involves systematic data discovery, automated Portable Document Format collection, table extraction, data cleaning and transformation, normalization, MySQL database implementation, and Entity Relationship Diagram development.

Our Solution: The developed system provides a fully normalized MySQL database comprising 21 relational tables, logically connected to represent the major entities of

the Bangladesh health sector. A conceptual (logical) Entity Relationship Diagram with approximately 40 entities and a physical database Entity Relationship Diagram are also produced, along with automated Extract-Transform-Load pipeline scripts intended for deployment to a version-controlled repository.

Evaluation: The solution is evaluated by running SQL (Structured Query Language) aggregate queries, validating record counts, checking foreign key integrity, and cross-referencing extracted data with original Portable Document Format source documents. This evaluation confirms a full, end-to-end connection between the source reports and the final database.

Significance, Limitations, and Future Work: This project demonstrates that unstructured government health data can be transformed into a structured, queryable format using systematic engineering approaches, giving health administrators and researchers a single point of access instead of hundreds of disconnected PDFs. A current limitation is that the database is built only from the reports collected during Sprint 1 and relies on manually verified source documents, so it does not yet cover the full range of Bangladesh’s health reporting. Future work includes extending coverage to additional health sectors, automating data updates from government portals, and building analytical dashboards on top of the database.

Chapter 1

Introduction

We are collecting information from various health-related government sources and organizing it into a single, structured, integrated database. The purpose of this project is to take scattered health information and organize it so that related information can be easily stored, retrieved, and queried. This report describes the project's background, problem, proposed system, development process, and related aspects, as our contribution to MODELBD — a large-scale, integrated Bangladesh sectoral database system developed collaboratively using the Scrum of Scrum methodology as part of the Database Systems Lab course at the Department of Computer Science and Engineering, Chittagong University.

The objective of this course is to develop an integrated health information database by applying the theories, methodologies, tools, and technologies learned in the Database Systems Lab course. Our group is responsible for the **Healthcare** sector.

1.1 Background and Motivation

Current State: Bangladesh's health sector produces a substantial volume of documentation annually through its Ministry of Health and Family Welfare, hospitals, and various health programs, covering disease surveillance, health information systems, and public health programs. These documents include annual health bulletins, hospital cancer registry reports, training reports, operational plans, budget documents, and statistical yearbooks. This kind of health information system is meant to support data analysis, interpretation, dissemination, and evidence-based planning and decision-making, but that intended use depends on the underlying information being organized and connected.

Problems Faced by Stakeholders: Despite the richness of this information, it remains locked inside Portable Document Format documents that are not machine-readable, and similar information is spread across many separate reports in inconsistent formats. Health administrators, researchers, and policymakers who need data-driven insights are forced to manually search through and cross-reference hundreds of documents to find and connect related information, which is both time-consuming and error-prone.

Our Approach: Rather than simply collecting these documents, our approach is to transform their contents into a structured database. We extract information from the source health documents, identify the relevant entities and their attributes, identify the relationships between those entities, and use this to build a conceptual Entity Relationship model, a relational model, and finally a normalized, implemented database.

Significance: Structuring this information matters because it turns fragmented, manually-searched documents into something that can be queried directly: related information becomes connected, retrieval becomes efficient, and the result provides a structured foundation for future analysis. Converting scattered health-related information into a structured and queryable database makes information retrieval, integration, and future analysis easier for the stakeholders described above.

1.2 Problem Statement

The problem addressed by this project is to identify, organize, and integrate the relevant health-related entity types, their attributes, and relationships from multiple heterogeneous government health documents into a structured, normalized relational database. Specifically, entities including Patient, HealthFacility, Disease, HealthWorker, CancerCase, Vaccination, Laboratory, MaternalHealth, AdministrativeRegion, and PopulationGroup are documented in government reports but lack structured, interconnected database representations. This project aims to design and implement a normalized database that represents these entities and the relationships among them in a consistent relational structure.

1.3 System Definition

A computerized system used to integrate and manage health-related information collected from multiple government documents, by organizing entities, attributes, and relationships — with well-defined entity relation types between them — into a normalized MySQL database accessible through standard SQL queries and supported by automated Extract-Transform-Load pipeline scripts, for efficient storage, retrieval, and querying.

1.4 System Development Process

The system is developed using an agile Scrum methodology organized in sprints. The development lifecycle includes the following phases:

1. **Data Discovery:** Identification and inventory of relevant government health documents.

2. **Data Acquisition:** Automated collection of Portable Document Format reports using Python scripts.
3. **Data Extraction:** Table and text extraction from Portable Document Format documents into Comma-Separated Values format.
4. **Data Cleaning and Transformation:** Standardization, deduplication, and normalization of extracted data.
5. **Entity Identification:** Identification of the major entities, and their attributes, present across the health domain.
6. **Relationship Identification:** Identification of the entity relation types (connections) between the identified entities.
7. **Conceptual Design:** Construction of the conceptual Entity Relationship model from the identified entities and relationships.
8. **Logical Design:** Conversion of the Entity Relationship model into a relational schema.
9. **Normalization:** Normalization to Third Normal Form to reduce redundancy and dependency anomalies.
10. **Database Implementation:** MySQL table creation and data loading.
11. **Validation:** SQL-based verification of data accuracy and integrity.
12. **Entity Relationship Diagram Development:** Conceptual (logical) and physical Entity Relationship Diagram generation.

1.5 Organization

Section 2 describes how the project resources and team roles are organized. Section 3 presents the Sprint 1 activities in detail, covering data discovery, acquisition, extraction, cleaning, database design, and Entity Relationship Diagram development. Section 4 explains the deployment and maintenance of the project repository. Section 5 reflects on intra- and inter-group collaboration. Section 6 offers comments on course pedagogy. Finally, Section 7 presents the conclusion and future work.

Chapter 2

Project Structure and Management

This chapter describes how our team is organized and how we plan, divide, track, and manage the project's work.

2.1 Resource Management

The project's resources are organized into three broad categories. The **human resources** are the four group members and the course supervisor. The **information resources** are the collected government health documents, extracted datasets, and reference materials that form the raw input to the database. The **software resources** are the tools used for version control, database development, and documentation, listed in Table 2.2. The project team consists of four members, and responsibilities are divided among them to maximize parallel progress while ensuring regular coordination, with each member owning a distinct area of the pipeline while remaining accountable to the shared Product Backlog described in Section 2.5.

2.2 Team Roles and Responsibilities

Assigning a distinct role to each member also gives the project accountability: if a particular task needs to be traced back to a person, for example, who designed the Entity Relationship diagram, or who maintained the Portable Document Format repository, the responsibility is clear from the table below.

Member	Role	Main Responsibilities
Md. Alamin Molla	Team Leader	Coordinates sprint planning, manages task distribution, and oversees database normalization and overall project progress
Sheikh Showkotul Islam Shakib	Data Collector	Identifies and collects government health reports, verifies document authenticity, and maintains the Portable Document Format repository
Ratul Hasan Nipu	Entity Relationship Diagram Designer	Designs the conceptual (logical) Entity Relationship Diagram (approximately 40 entities) and the physical database Entity Relationship Diagram based on the normalized schema
Jahid Alam Ridoy	Script Writer	Develops Python scripts for automated Portable Document Format downloading, table extraction, data cleaning, and MySQL import utilities

Table 2.1: Team Roles and Responsibilities

2.3 Tools Used

Each tool below serves a specific purpose in the workflow: GitHub provides version control and a shared history of the codebase so that changes can be tracked and previous versions recovered; Python, Pandas, and Regular Expressions handle the data-processing side (downloading, extracting, and cleaning); MySQL is the target database system; and Mermaid/L^AT_EX TikZ are used to visualize the Entity Relationship diagrams once the schema is finalized.

Tool/Technology	Purpose
GitHub	Version control, code repository, Extract-Transform-Load pipeline deployment
Python	Portable Document Format downloading, data extraction, data cleaning scripts
Pandas	Data manipulation and transformation
MySQL	Relational database implementation
Mermaid / $\text{\LaTeX}$ TikZ	Entity Relationship Diagram visualization
Linux Terminal	File management, bulk processing, command-line tools
Regular Expressions	Pattern matching during data cleaning

Table 2.2: Tools and Technologies

2.4 Scrum Structure

Scrum is an agile project-management framework in which a large project is broken into smaller tasks, and progress is tracked regularly through short cycles of work called sprints. Briefly, our project follows a three-level Scrum of Scrum hierarchy. At Level 1, our group independently develops the Healthcare sector database. Level 2, in which our group’s outputs will be integrated with the other sector groups, and Level 3, in which the combined sectors form the complete MODELBD system, have not yet taken place at the time of writing this report.

Within our group, we maintain a **Product Backlog** listing all major tasks (Section 2.5). Each sprint begins with a **Sprint Planning** meeting, in which a subset of the Product Backlog is selected as that sprint’s Sprint Backlog; progresses through **Daily Stand-ups**, where progress and blockers are discussed; and concludes with a **Sprint Review**, where completed work is checked against the sprint’s goals.

2.5 Product Backlog Summary

The Product Backlog is the prioritized list of all tasks, features, and requirements needed to complete the project; without it, task ownership, sequencing, and completion status would be difficult to track across four people. The Product Backlog for Sprint 1 includes the following items:

1. Identify and inventory health data sources from government portals
2. Develop automated Portable Document Format downloader

3. Extract tables from Portable Document Format reports into Comma-Separated Values format
4. Clean and standardize extracted datasets
5. Design normalized database schema
6. Implement MySQL database with 21 tables
7. Validate data accuracy using SQL queries
8. Develop conceptual (logical) Entity Relationship Diagram (approximately 40 entities)
9. Generate physical database Entity Relationship Diagram
10. Document all processes

Chapter 3

Sprint 1

Following our Scrum structure, this chapter documents Sprint 1 as a complete record of what was planned, what we did, the problems we faced, how we solved them, and what we produced. The **Sprint Goal** for Sprint 1 is to take the Healthcare sector from raw, unstructured government reports to a validated, normalized MySQL database with accompanying Entity Relationship diagrams. Sprint 1 covers the complete development cycle from data discovery to database implementation and Entity Relationship Diagram generation over six weeks.

3.1 Sprint Backlog and Overview

Week	Task	Duration	Outcome
Week 1	Data Collection & Source Identification	5–6 Days	100+ reports inventoried
Week 2	Automated Portable Document Format Collection	4–5 Days	Automated repository created
Week 3	Portable Document Format Processing & Table Extraction	6–7 Days	Hundreds of Comma-Separated Values files generated
Week 4	Data Cleaning & Transformation	4–5 Days	Database-ready CSVs produced
Week 5	Database Design & Normalization	3 Days	Fully normalized schema
Week 6	MySQL Implementation, Validation & Entity Relationship Diagram Development	2–3 Days + 5 Days	21 tables created, Entity Relationship Diagram and validated database produced

Table 3.1: Sprint 1 Task Summary

3.2 Week 1: Data Collection and Source Identification

The sprint commences with the identification of reliable health-related datasets from Bangladesh Government publications. The initial expectation is that structured datasets

would be readily available in Excel or database format. However, the data collection stage reveals numerous challenges.

3.2.1 Challenges Faced

- Health information is distributed across multiple government websites, annual health bulletins, hospital cancer registry reports, training reports, and statistical yearbooks.
- Many document download links are inactive or broken.
- Most datasets exist only as Portable Document Format documents rather than machine-readable files.
- Similar information appears across multiple documents with different naming conventions, reporting periods, and classifications.
- The same health program is sometimes referred to by its full name and sometimes by an abbreviation, causing confusion during integration.

3.2.2 Approach

To address these issues, we manually review available reports, verify document authenticity, categorize documents by topic, and prepare an organized inventory before proceeding.

3.2.3 Outcome

- More than 100 health-related reports are identified.
- Relevant datasets are shortlisted.
- A source inventory is prepared for automated collection.

3.3 Week 2: Automated Portable Document Format Collection and Repository Creation

Once relevant reports are identified, manually downloading each file becomes impractical. Therefore, automated Portable Document Format collection is implemented using Python.

3.3.1 Challenges Faced

- Different websites use different web-address structures and file naming conventions.

- Some files contain special characters in their names.
- Incomplete downloads occur due to temporary network interruptions.
- Without proper organization, hundreds of downloaded files would become unmanageable.

3.3.2 Tools Used

Python, Required Library, Operating System and File Handling Modules, Linux Terminal.

3.3.3 Performance

- Small batch download: 10–20 seconds
- Large batch download: 2–5 minutes

3.3.4 Outcome

- An automated Portable Document Format repository is created.
- Hundreds of reports are downloaded successfully.
- Manual effort is reduced significantly.

3.4 Week 3: Portable Document Format Processing and Table Extraction

This is the most challenging phase of Sprint 1. Although the reports appear visually organized, extracting usable data proves extremely difficult due to complex Portable Document Format layouts.

3.4.1 Challenges Faced

- Tables often extend across multiple pages, creating the impression that data is missing.
- Some table entries are broken into several rows during extraction, causing columns to shift.
- Many extracted tables lose their headers.
- Certain pages contain scanned content where traditional extraction methods perform poorly.

3.4.2 Tools Used

Python, Pandas, Comma-Separated Values Processing, Regular Expressions, and Linux command-line tools including `grep`, `sed`, `awk`, `head`, `tail`, `find`, and `wc`.

3.4.3 Performance

- Single Portable Document Format extraction: 2–10 seconds
- Bulk extraction: 5–20 minutes

3.4.4 Outcome

- Hundreds of structured Comma-Separated Values files are generated.
- Cancer registry datasets are reconstructed.
- Missing sections are successfully recovered.

3.5 Week 4: Data Cleaning, Transformation and Categorization

After extraction, the generated Comma-Separated Values files require extensive cleaning before they can be loaded into a database.

3.5.1 Data Quality Issues Identified

- Missing values across multiple fields.
- Duplicate records from overlapping reports.
- Inconsistent spellings and abbreviations of health program names.
- Numeric fields occasionally containing symbols instead of numeric values.
- Multi-line records requiring reconstruction.

3.5.2 Tools Used

Python, Pandas, Regular Expressions, and spreadsheet-based spot checks for manual verification of edge cases.

3.5.3 Outcome

- Clean datasets are produced.
- Standardized naming conventions are established.

- Database-ready Comma-Separated Values files are prepared.

3.6 Week 5: Database Design and Normalization

3.6.1 Normalization

Once clean datasets are available, normalization principles are applied to eliminate redundancy and anomalies.

First Normal Form: Repeated groups and multi-valued attributes are removed, for example, splitting the disease-specific attributes (Dengue type, COVID-19 test result, HIV status, and so on) out of a single “Disease” record into their own subtype tables.

Second Normal Form: Partial dependencies are eliminated; attributes that depend on only part of a composite key (for example, facility details inside a bed-allocation record) are moved into their own table (HEALTH_FACILITY).

Third Normal Form: Transitive dependencies are removed, for example, `RegionName` and `RegionType` are kept only in `ADMINISTRATIVE_REGION` rather than being repeated wherever a region is referenced.

3.6.2 Worked Example: Normalizing the Patient Table (Unnormalized Form to Boyce–Codd Normal Form)

To make the normalization process concrete, this subsection walks a single source relation, a flat “Patient Care Record” built from the `Patient` entity together with the attributes reachable through its *Assigned To* relationship to `HospitalBed` (and, through the bed, to `HealthFacility`) and its *Receives* relationship to `MaternalHealth`, all the way from an unnormalized starting table through First Normal Form, Second Normal Form, Third Normal Form, and finally Boyce–Codd Normal Form. The parallel *Gives Birth To* relationship to `Newborn` is left out of this particular walkthrough because combining two independent one-to-many relationships (maternal-health episodes and newborns) in a single flat table would introduce a multi-valued dependency, which is a Fourth Normal Form concern rather than a Boyce–Codd Normal Form one; the same four-step method applies to it directly.

The following working assumptions and functional dependencies are used throughout this example:

- At any given time, a patient occupies exactly one hospital bed, and each bed belongs to exactly one health facility.

- A patient may have several maternal-health episodes over time, each beginning on a different date.
- **NationalID** is unique per patient and therefore serves as an alternate candidate key alongside the surrogate **PatientID**.

FD	Functional Dependency
FD1	$\text{PatientID} \rightarrow \text{FullName}, \text{DateOfBirth}, \text{Gender}, \text{NationalID}, \text{BedID}$
FD2	$\text{NationalID} \rightarrow \text{PatientID}, \text{FullName}, \text{DateOfBirth}, \text{Gender}, \text{BedID}$ (alternate key)
FD3	$\text{BedID} \rightarrow \text{BedType}, \text{Status}, \text{FacilityID}$
FD4	$\text{FacilityID} \rightarrow \text{FacilityName}$

Table 3.2: Functional Dependencies Used in the Patient Normalization Example

StartDate is a genuine multi-valued fact of a patient (one value per maternal-health episode) and is not determined by any of the functional dependencies above.

Description: A relation is unnormalized when a single cell is allowed to hold more than one value. Here, the **StartDate** column holds a comma-separated list of maternal-health episode dates for patients P001 and P003, which is not an atomic value, so this table is not yet even in First Normal Form.

The repeating group is eliminated by giving each maternal-health episode its own row and repeating the patient-level data as needed. Because **PatientID** now repeats for patients with several episodes, and **StartDate** alone is not unique across different patients, the primary key becomes the composite $\{\text{PatientID}, \text{StartDate}\}$. Every value is now atomic, but **FullName** through **FacilityName** still repeat on every row for the same patient; that redundancy is exactly what Second Normal Form removes next.

Second Normal Form

Patient_2NF (Key: PatientID)		
Column	Key	Note
PatientID	PK	Surrogate patient identifier
FullName, Date-Of-Birth, Gender	–	Depend on PatientID (FD1)
NationalID	–	Depends on PatientID (FD1); also an alternate key (FD2)
BedID	–	Depends on PatientID (FD1)
BedType, Status	–	Depend on PatientID only transitively, via BedID (FD3)
FacilityID, FacilityName	–	Depend on PatientID only transitively, via BedID and FacilityID (FD3, FD4)

Table 3.5: Second Normal Form – Table 1

MaternalHealth_2NF (Key: {PatientID, StartDate})		
Column	Key	Note
PatientID	PK / FK	References Patient_2NF.PatientID
StartDate	PK	Start date of one maternal-health episode

Table 3.6: Second Normal Form – Table 2

Description: By functional dependencies FD1–FD4, `FullName` through `FacilityName` all depend on `PatientID` alone, only part of the composite key `{PatientID, StartDate}`, which is a partial dependency and a violation of Second Normal Form. Splitting the relation into `Patient_2NF` (everything that depends on `PatientID` alone) and `MaternalHealth_2NF` (the composite-key facts) removes this violation without losing any data: `MaternalHealth_2NF.PatientID` still links every episode back to its patient. `Patient_2NF` is still carrying `BedType`, `Status`, `FacilityID`, and `FacilityName`, which really describe the bed and the facility rather than the patient; that transitive redundancy is what Third Normal Form removes next.

Third Normal Form

Patient (Key: PatientID; alternate key: NationalID)		
Column	Key	Note
PatientID	PK	Surrogate patient identifier
FullName, Date-OfBirth, Gender	–	FD1
NationalID	–	FD1 / FD2, alternate candidate key
BedID	FK	FD1, references Bed.BedID

Table 3.7: Third Normal Form – Patient

Bed (Key: BedID)		
Column	Key	Note
BedID	PK	Physical bed identifier
BedType, Status	–	FD3
FacilityID	FK	FD3, references Facility.FacilityID

Table 3.8: Third Normal Form – Bed

Facility (Key: FacilityID)		
Column	Key	Note
FacilityID	PK	Health facility identifier
FacilityName	–	FD4

Table 3.9: Third Normal Form – Facility

MaternalHealth (Key: {PatientID, StartDate})		
Column	Key	Note
PatientID	PK / FK	References Patient.PatientID
StartDate	PK	Start date of one maternal-health episode

Table 3.10: Third Normal Form – MaternalHealth

Description: In *Patient_2NF*, the non-key attribute *BedID* determines *BedType* and *Status* (FD3), and the non-key attribute *FacilityID* determines *FacilityName* (FD4); both are transitive dependencies (a non-key attribute determining another non-key attribute). Splitting *Patient_2NF* further into *Patient*, *Bed*, and *Facility* removes these transitive dependencies; *MaternalHealth* is unaffected, since it has no non-key

attributes to begin with. The schema now has four tables, and every non-key attribute depends directly on its own table's key.

Boyce–Codd Normal Form Verification

Table	Determinant $\rightarrow$ Dependent(s)	Determinant is a Candidate Key?	Verdict
Patient	PatientID $\rightarrow$ FullName, DateOfBirth, Gender, NationalID, BedID	Yes (primary key)	Holds
Patient	NationalID $\rightarrow$ PatientID, FullName, DateOfBirth, Gender, BedID	Yes (alternate key)	Holds
Bed	BedID $\rightarrow$ BedType, Status, FacilityID	Yes (primary key)	Holds
Facility	FacilityID $\rightarrow$ FacilityName	Yes (primary key)	Holds
MaternalHealth	No non-key attribute exists	Not applicable	Holds (trivially)

Table 3.11: Boyce–Codd Normal Form Verification Across All Four Tables

Description: A relation is in Boyce–Codd Normal Form if, for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey. Unlike Third Normal Form, Boyce–Codd Normal Form makes no exception for attributes that belong to a candidate key; it is checked on every table without exception. Every determinant found across all four tables is already a candidate key of the table it appears in, so no Boyce–Codd Normal Form violation exists, and no further decomposition is required: the Third Normal Form schema already satisfies Boyce–Codd Normal Form.

Table	Primary Key	Foreign Key $\rightarrow$ References
Patient	PatientID	BedID $\rightarrow$ Bed.BedID
Bed	BedID	FacilityID $\rightarrow$ Facility.FacilityID
Facility	FacilityID	(none)
MaternalHealth	{PatientID, StartDate}	PatientID $\rightarrow$ Patient.PatientID

Table 3.12: Final Boyce–Codd Normal Form Schema for the Patient Care Record

Description: These four tables, *Patient*, *Bed*, *Facility*, and *MaternalHealth*, are all lossless-join and dependency-preserving decompositions of the original Patient Care

Record: no data and no functional dependency from the source Entity Relationship Diagram has been lost along the way. This worked example illustrates, at the level of a single entity, the same normalization discipline that is applied across the full 21-table schema described in Table 3.2.

3.6.3 Outcome

A fully normalized relational schema is produced with reduced redundancy and improved consistency.

3.7 Week 6: MySQL Implementation, Validation and Entity Relationship Diagram Development

Twenty-one normalized tables are created and linked using primary keys and foreign keys in MySQL. Key implementation challenges include datatype mismatches, foreign key dependency ordering, and null value handling. These are resolved using custom SQL scripts and Python import utilities.

Table No.	Table Name	Description
1	ADMINISTRATIVE_REGION	Administrative divisions/regions of Bangladesh (region name, type, population)
2	POPULATION_GROUP	Population figures for a region, broken down by group
3	HEALTH_FACILITY	Health facility records (type, name) linked to an administrative region
4	HOSPITAL_BED	Bed inventory and status per health facility
5	LABORATORY	Laboratory records linked to a health facility
6	TELEMEDICINE_CENTER	Telemedicine centers offering remote consultations to patients
7	DESIGNATION	Health workforce job designations
8	HEALTH_WORKER	Health worker records, linked to a facility and a designation
9	PATIENT	Core patient records (name, date of birth, gender, National Identification Number)
10	DISEASE	Catalog of tracked diseases with International Classification of Diseases codes
11	DENGUE	Dengue-specific case details (subtype of Disease)
12	COVID19	COVID-19-specific case details (subtype of Disease)
13	MEASLES	Measles-specific case details (subtype of Disease)
14	HIV	HIV-specific case details (subtype of Disease)
15	DIARRHEA	Diarrhea-specific case details (subtype of Disease)
16	CANCER_CASE	Cancer case records (type, stage, test result) tested at a laboratory
17	BIOPSY	Biopsy procedures linked to a cancer case
18	VACCINATION	Vaccine catalog and patient vaccination records
19	MATERNAL_HEALTH	Maternal health records linked to a patient
20	NEWBORN	Newborn records linked to a mother's maternal health record
21	MALNUTRITION	Malnutrition assessment records (height, Body Mass Index, type)

Table 3.13: 21 Normalized MySQL Database Tables

3.7.1 Sample Normalized Tables (1NF to 3NF)

To illustrate how the normalization principles described in Section 3.6.2 apply across the schema, this subsection presents five representative entities from the implemented database, populated with sample Bangladeshi health-sector data and accompanied by an explanation of how each satisfies First, Second, and Third Normal Form.

HealthFacility

Keys: FacilityID (Primary Key), RegionID (Foreign Key)

FacilityID	FacilityName	FacilityType	RegionID
FAC001	Dhaka Medical College Hospital	Tertiary Medical College Hospital	REG-DHK
FAC002	Chittagong Medical College Hospital	Tertiary Medical College Hospital	REG-CTG
FAC003	Savar Upazila Health Complex	Upazila Health Complex (UHC)	REG-DHK
FAC004	Sylhet MAG Osmani Medical College Hospital	Tertiary Medical College Hospital	REG-SYL
FAC005	Gazipur Sadar Community Clinic	Community Clinic (CC)	REG-GZP

Table 3.14: HealthFacility – Sample Normalized Data

Normalization Explanation (up to 3NF): The HealthFacility entity satisfies First Normal Form by enforcing atomic values across all columns and maintaining unique row identification through FacilityID. It fulfills Second Normal Form because all non-key attributes (FacilityName and FacilityType) depend fully on the entire primary key FacilityID rather than on any partial key. Third Normal Form is achieved by eliminating transitive dependencies: regional details such as district or division names are decoupled into a separate Region entity, leaving only the foreign key RegionID to maintain relational integrity.

Table Content Summary: This table serves as the primary registry for medical infrastructure across Bangladesh. It stores essential information regarding healthcare institutions, categorizing them into administrative facility tiers ranging from grassroots Community Clinics and Upazila Health Complexes to major tertiary Medical College Hospitals.

HospitalBed

Keys: BedID (Primary Key), FacilityID (Foreign Key)

BedID	BedType	Status	FacilityID
BED-101	ICU Bed	Occupied	FAC001
BED-102	CCU Bed	Available	FAC001
BED-201	General Male Ward Bed	Occupied	FAC002
BED-301	Maternity Ward Bed	Available	FAC003
BED-401	Pediatric Isolation Bed	Maintenance	FAC004

Table 3.15: HospitalBed – Sample Normalized Data

Normalization Explanation (up to 3NF): HospitalBed achieves First Normal Form by containing a single scalar value in every cell and eliminating repeating groups. Second Normal Form compliance is established because the single primary key **BedID** uniquely determines both **BedType** and **Status**, avoiding partial dependency issues. To meet Third Normal Form, facility-level metadata, such as overall hospital capacity or location, is omitted, relying solely on the **FacilityID** foreign key to reference facility details and preventing transitive functional dependencies.

Table Content Summary: This table tracks physical inpatient assets and real-time bed availability within healthcare facilities. It provides operational visibility into critical medical resource categories such as Intensive Care Unit (ICU) beds, Coronary Care Unit (CCU) beds, general ward units, and maternity facilities.

Designation

Keys: DesignationID (Primary Key)

DesignationID	DesignationName
DES001	Assistant Surgeon (BCS Health)
DES002	Senior Staff Nurse
DES003	Sub-Assistant Community Medical Officer (SACMO)
DES004	Health Assistant (HA)
DES005	Medical Officer (Civil Surgeon Office)

Table 3.16: Designation – Sample Normalized Data

Normalization Explanation (up to 3NF): The Designation table achieves First Normal Form by ensuring all entries are indivisible scalar values. It satisfies Second and Third Normal Form directly, as a lookup dictionary containing a single candidate key (**DesignationID**) that directly determines **DesignationName**. Because there are no non-key dependencies or multi-attribute keys, partial and transitive dependencies are naturally impossible, confirming Third Normal Form compliance.

Table Content Summary: This table functions as a standardized reference lookup for official healthcare job titles and cadre posts defined by the Bangladesh Directorate General of Health Services (DGHS). It categorizes clinical and administrative personnel ranging from BCS Health Cadre Assistant Surgeons to field-level Health Assistants.

Disease

Keys: ICDCode (Primary Key)

ICDCode	DiseaseName
A90	Dengue Hemorrhagic Fever
A00.9	Cholera (Diarrheal Infection)
U07.1	COVID-19 Infection
B05.9	Measles
A15.0	Tuberculosis (Pulmonary TB)

Table 3.17: Disease – Sample Normalized Data

Normalization Explanation (up to 3NF): The **Disease** entity adheres to First Normal Form through atomic International Classification of Diseases (ICD) code and disease name entries. It achieves Second Normal Form because the primary key **ICDCode** fully functionally determines the **DiseaseName** attribute. Third Normal Form is verified because there are no transitive functional relationships between non-key attributes; the table isolates clinical disease cataloging from treatment or facility assignment data.

Table Content Summary: This table maintains a standardized master record of medical conditions and infectious diseases prevalent in Bangladesh. It uses World Health Organization (WHO) International Classification of Diseases (ICD) codes to record illnesses such as Dengue Fever, Cholera, Tuberculosis, and COVID-19.

HealthWorker

Keys: WorkerID (Primary Key), FacilityID (Foreign Key), RegionID (Foreign Key), DesignationID (Foreign Key)

WorkerID	WorkerName	Gender	FacilityID	RegionID	DesignationID
HW001	Dr. Md. Rafiqul Islam	Male	FAC001	REG-DHK	DES001
HW002	Nusrat Jahan Begum	Female	FAC001	REG-DHK	DES002
HW003	Abul Kalam Azad	Male	FAC002	REG-CTG	DES003
HW004	Taslima Akter	Female	FAC003	REG-DHK	DES004
HW005	Dr. Farhana Chowdhury	Female	FAC004	REG-SYL	DES005

Table 3.18: HealthWorker – Sample Normalized Data

Normalization Explanation (up to 3NF): HealthWorker satisfies First Normal Form by storing individual atomic attributes for each medical employee. Second Normal Form is met because all personal attributes (WorkerName, Gender) depend entirely on the single primary key WorkerID. The structure achieves Third Normal Form by extracting external entities, such as facility details, regional administrative zones, and job titles, into foreign keys (FacilityID, RegionID, DesignationID), eliminating transitive functional dependencies and reducing update anomalies.

Table Content Summary: This table stores profile and deployment records for medical and health personnel across Bangladesh. It links individual healthcare professionals, including doctors, nurses, and community health officers, to their assigned health facility, region, and official professional designation.

Relational Schema and Normalization Compliance Summary:

- **First Normal Form:** All tables feature unique record identifiers and atomic scalar column values with no repeating groups.
- **Second Normal Form:** All non-key attributes are fully dependent on their primary key, eliminating partial functional dependencies.
- **Third Normal Form:** Transitive dependencies are eliminated across all entities by abstracting reference data into dedicated lookup tables linked via foreign keys.

3.8 Logical Entity Relationship Diagram Development

Before generating the final database Entity Relationship Diagram, a conceptual, also referred to as logical, Entity Relationship Diagram is developed from the health reports. Approximately **21 major entities** are identified and analyzed from the source documents, and these entities are carried through into the physical database as:

Sample Identified Entities			
Patient	Health Facility	Disease	Cancer Case
Health Worker	Laboratory	Telemedicine Center	Hospital Bed
Vaccination	Maternal Health	Newborn	Malnutrition
Administrative Region	Population Group	Designation	Biopsy
Dengue	Covid19	Measles	HIV / Diarrhea

Table 3.19: Sample Entities from Conceptual Entity Relationship Diagram

The diagram is a detailed Entity Relationship Diagram (ERD) for a 'Project: Health' database. It features a central 'Disease' entity (rectangle) with attributes like 'Fever', 'DiagnosisDate', 'Symptoms', 'DiagnosisID', 'DiseaseName', 'ICDCode', and 'Temperature'. This central entity is connected to several other entities: 'Measles', 'HIV', 'Dengue', 'Covid19', 'Diarrhea', and 'PopulationGroup'. Each of these entities has its own set of attributes, such as 'BirthDate', 'Gender', 'NationalID', 'Fullname', 'Status', 'Health', 'FacilityID', 'RegionID', 'FacilityType', 'FacilityName', 'LabID', 'LabName', 'CancerType', 'CancerID', 'CancerStage', 'TestResult', 'AdmissionDate', 'DischargeID', 'Age', 'Gender', 'DiseaseID', 'VaccineID', 'VaccineName', 'MainnutritionID', 'MainnutritionType', and 'BMI'.

Relationships (diamonds) connect these entities, often with cardinalities (1, M, N). Key relationships include 'Receives' (MaternalHealth to Patient), 'Gives Birth To' (Patient to Newborn), 'Infected With' (Newborn to Measles), 'Assigned' (Measles to Designation), 'Suffers From' (Designation to Measles), 'Treats' (Measles to HospitalBed), 'Operates' (HospitalBed to HealthFacility), 'Has' (HealthFacility to TelemedicineCenter), 'Consults' (TelemedicineCenter to CancerCase), 'Undergoes' (CancerCase to Vaccination), and 'Affected By' (Vaccination to Mainnutrition). Foreign keys (dashed ovals) are used to link attributes to their parent entities, such as 'PatientID (FK)' for Patient, 'MotherID (FK)' for Newborn, 'DesignationID (FK)' for Designation, 'RegionID (FK)' for PopulationGroup, 'DiseaseID (FK)' for Measles, 'HIVStatus (FK)' for HIV, 'DiseaseID (FK)' for Dengue, 'Covid19ID (FK)' for Covid19, 'DiagnosisID (FK)' for Diarrhea, 'DiseaseID (FK)' for Disease, 'LabID (FK)' for Laboratory, 'PatientID (FK)' for Biopsy, 'CancerCaseID (FK)' for Biopsy, 'VaccineID (FK)' for Vaccination, and 'MainnutritionID (FK)' for Mainnutrition.

3.9 Entity-Relationship (ER) Diagram Description

The proposed Health System Entity-Relationship Diagram (ERD) is designed to model the relationship between administrative regions, healthcare facilities, personnel, patients, and various disease/condition monitoring records.

3.9.1 Entities and Attribute Details

AdministrativeRegion Contains geographical region details.

- **RegionID** (Primary Key), RegionName, Population, RegionType

HealthFacility Represents clinics, hospitals, or centers situated in a region.

- **FacilityID** (Primary Key), FacilityType, FacilityName, RegionID (Foreign Key)

HealthWorker Medical staff working at a specific healthcare facility.

- **WorkID** (Primary Key), WorkerName, FacilityID (Foreign Key), Gender

Laboratory Facilities associated with health centers for testing.

- **LabID** (Primary Key), LabName

HospitalBed Beds available within a health facility.

- **BedID** (Primary Key), FacilityID (Foreign Key), BedType, Status

Patient General information of individuals receiving medical attention.

- **PatientID** (Primary Key), FullName, DateOfBirth, Gender, NationalID

MaternalHealth Tracks pregnancy and maternal health records linked to patients.

- **MotherID** (Primary Key), PatientID (Foreign Key), StartDate

Newborn Information regarding newborn children.

- **NewbornID** (Primary Key), MotherID (Foreign Key), BirthDate

TelemedicineCenter Handles remote consultations provided to patients.

- **CenterID** (Primary Key), ConsultationID, PatientID (Foreign Key)

PopulationGroup Demographic grouping within a specific region.

- **PopulationGroupID** (Primary Key), RegionID (Foreign Key)

Designation Staff designations and roles.

- **DesignationID** (Primary Key), DesignationName

3.9.2 Medical Conditions and Disease Tracking Entities

Malnutrition **MalnutritionID** (PK), Height, BMI, MalnutritionType

Vaccination **VaccineID** (PK), VaccineName

CancerCase **CancerID** (PK), CancerType, TestResult, CancerStage

Biopsy **BiopsyID** (PK), CancerCaseID (FK), ProcedureDate

Disease **DiseaseID** (PK), DiseaseName, ICDCCode

Covid19 **CovidID** (PK), TestResult, Temperature

Measles **MeaslesID** (PK), Symptoms, Fever, DiagnosisDate

HIV **HIVID** (PK), DiagnosisDate, HIVStatus, TestResult

Dengue **DengueID** (PK), Age, Gender, DengueType

Diarrhea **DiarrheaID** (PK), Symptoms, TestResult, AdmissionDate

3.9.3 Relationships Between Entities

- **AdministrativeRegion** *contains* **HealthFacility** and **PopulationGroup**.
- **HealthFacility** *employs* **HealthWorker**, *has* **Laboratory**, and *provides* **HospitalBed**.
- **Patient** *is assigned to* a **HospitalBed**, *undergoes* **Vaccination**, and *receives* telemedicine consultations.
- **MaternalHealth** *gives birth to* **Newborn**.
- **CancerCase** *requires* **Biopsy**.

3.10 Data Validation

After database construction, extensive validation is performed using SQL queries to verify record counts, relationship consistency, aggregate totals, and statistical accuracy. Results are cross-checked against original source documents.

3.11 Final Sprint 1 Deliverables

1. Health Data Repository (Portable Document Format collection)
2. Automated Portable Document Format Collection System (Python scripts)
3. Extracted Comma-Separated Values Dataset Collection

4. Data Cleaning and Transformation Scripts
5. MySQL Database with 21 normalized tables
6. Relationship and Entity Documentation
7. Conceptual (Logical) Entity Relationship Diagram (approximately 40 Entities)
8. Physical Database Entity Relationship Diagram (21 Tables)
9. Project Workflow Documentation

Chapter 4

Deployment and Maintenance

4.1 Repository Structure

All Extract-Transform-Load pipeline code, data cleaning scripts, extracted Comma-Separated Values datasets, SQL schema files, and documentation are deployed to a public GitHub repository.

- **Repository Web Address:** https://github.com/Ratul-debug/Health_DB_Project

The repository is organized into the following directories:

- `/extracted_tables/`, Extracted Comma-Separated Values files
- `/metadata/`, Metadata management and execution logging files
- `/pdfs/`, Downloaded Portable Document Format reports
- `ER_DIAGRAM.png`, Entity Relationship Diagram visual image asset
- `Health_ER_Diagram_.pdf`, Main relational design schema document
- `health_data.xlsx`, Original source spreadsheet matrix file

4.2 How to Run the Pipeline

1. Clone the repository from GitHub.
2. Install required Python packages: `pip install -r requirements.txt`
3. Run the Portable Document Format downloader: `python scripts/downloader.py`
4. Run the extractor: `python scripts/extractor.py`
5. Run the cleaner: `python scripts/cleaner.py`
6. Import into MySQL: `mysql -u root -p < sql/schema.sql`
7. Validate the database: run the SQL validation scripts in `/sql/validation/`.

4.3 Maintenance

The database and scripts are designed for incremental updates. New government reports can be added to the source list and reprocessed through the pipeline without modifying the schema, provided the data conforms to the established entity structure.

Chapter 5

Collaboration

5.1 Intra-Group Collaboration

Within our group, collaboration is managed through regular meetings, including daily and monthly progress check-ins, and a shared GitHub codebase. Task assignments are tracked using the sprint backlog, and progress is reviewed at the end of each work session to reduce collaboration redundancy and keep meeting time focused.

A key focus of this intra-group collaboration is coordinating with the **Obsidians**, who are working on the same database project. Since both of our teams are independently developing different parts of the database, we communicate closely to identify overlapping entities and attributes. Although this cross-team collaboration is targeted, these periodic discussions are crucial to minimize data redundancy, avoid designing duplicate attributes or similar components, and ensure seamless consistency and compatibility between our two databases.

Overall, intra-group collaboration is productive. Splitting responsibilities by role, team leadership, data collection, Entity Relationship Diagram design, and script writing, means that each member can work in parallel on a well-defined piece of the project while still depending on the others' outputs, which keeps everyone engaged with the bigger picture. The most challenging moments arise during the data cleaning phase, when extracted Comma-Separated Values (CSV) files from different sources do not follow the same column structure; reconciling these requires several back-and-forth discussions and a shared agreement on naming conventions before the database design can proceed. Regular short check-ins help catch these mismatches and cross-team entity overlaps early, rather than after the schema has already been finalized.

5.2 Inter-Group Collaboration

At the time of writing this report, the inter-level Scrum of Scrum sessions with the other sector groups have not yet taken place. Our group has so far focused on completing the Healthcare sector deliverables independently, and inter-group integration with the other six MODELBD sectors (Agriculture, Energy, and others) is expected to begin in a subsequent phase of the course. This section will be expanded with our experiences once those sessions occur.

5.3 Good Memory

The most rewarding moment of Sprint 1 is watching the first successful end-to-end run of the pipeline: a batch of previously unreadable Portable Document Format health bulletins goes in, and clean, query-able rows land in MySQL tables that match our normalized schema. It confirms that the whole chain, from messy government PDFs to a working relational database, actually holds together.

5.4 Bad Memory

The most difficult stretch is during the data cleaning phase (Week 4), when we discover that Comma-Separated Values files extracted from different cancer-registry reports use inconsistent column orders and naming for the same fields. It takes several rounds of manual comparison and a team discussion to agree on a single standardized schema before we can safely merge the datasets.

Chapter 6

Comments on Course Pedagogy

Aspect	Opportunity / Strength	Limitation / Suggestion
Scrum Methodology	Provides real industry experience in agile project management	Initial learning curve for students unfamiliar with Scrum
Collaborative Development	Working on a shared system builds teamwork and integration skills	Coordination overhead increases with group size
Real Data Usage	Using actual government data makes the project meaningful	Real data is messy and requires significant preprocessing time
Industry Relevance	Extract-Transform-Load pipelines, version control, and normalization are directly applicable to industry	A short introductory session on Git/GitHub workflows early in the course would help students who have not used version control collaboratively before
Data Quality & Performance	Working with real, messy government data gives hands-on experience in validation, integrity-checking, and query optimization, skills directly transferable to industry data engineering roles	Proper data validation and database optimization techniques should be applied more systematically (for example, automated constraint checks, indexing strategy) to improve long-term system reliability and performance

Table 6.1: Course Pedagogy: Opportunities and Limitations

6.1 Lessons Learned

Through this project, we gain practical experience in several areas beyond standard coursework:

- **Skills Learned:** Git and GitHub (version control and collaborative workflows), Python (scripting for automation and data processing), Extract-Transform-Load

pipeline development, and Scrum (agile project management), all applied at the scale of a complete, working “tiny project” delivered end-to-end.

- **Industry Processes:** Real-world data is rarely structured. Extract-Transform-Load pipelines are fundamental to any data-driven system.
- **Collaboration:** Clear task division and version control are essential when multiple people work on the same codebase.
- **Conflict Resolution:** Differences in naming conventions and data interpretations require negotiation and consensus.
- **Management:** Sprint-based planning helps keep the project on track despite the large volume of work.
- **Database Design:** We learn practical database design using Entity Relationship diagrams, translating real-world health entities and relationships into a normalized schema.
- **SQL Proficiency:** Our SQL querying and database management skills improve substantially through writing validation queries, joins, and integrity checks.
- **Teamwork and Task Distribution:** Dividing tasks across data collection, extraction, cleaning, and database design gives us direct experience in team-based software delivery.
- **Problem-Solving:** Implementing the database, from schema design to handling malformed data, strengthens our practical problem-solving skills.
- **Documentation and Presentation:** Preparing this report and the accompanying ERDs enhances our ability to document and present technical work clearly.

6.2 Suggestions for Future Batches

- Providing students with a sample cleaned dataset in the first week would reduce time spent on data quality issues and allow more focus on database design.
- At the beginning of the project, a clear and detailed guideline should be provided by the course teacher or project mentor regarding the project objectives, requirements, and expected outcomes.
- Sufficient time should be allocated for project planning, database design, implementation, testing, and documentation.
- Regular consultation sessions with the mentor can help students overcome challenges more effectively.

- Sharing a list of known-reliable government data sources at the start of the course would reduce the time groups spend on broken links and unreliable mirrors during data discovery.
- Scheduling the inter-level Scrum of Scrum sessions earlier, in parallel with Sprint 1 rather than strictly after it, would let groups align on shared entities (such as **District/Division**) before each group finalizes its own schema independently.
- Future versions of the project may include a graphical user interface, enhanced security features, and advanced reporting capabilities.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project addresses the problem of inaccessible and unstructured health data from Bangladesh government sources. The developed solution is a fully normalized MySQL database comprising 21 relational tables covering the core entities of the Bangladesh healthcare ecosystem, supported by automated Extract-Transform-Load pipeline scripts and comprehensive ERDs.

Starting from over 100 scattered Portable Document Format documents and ending with a structured, validated, and queryable relational database, the project demonstrates that systematic data engineering, including automated collection, extraction, cleaning, normalization, and validation, can transform unstructured government data into a professionally usable information system.

The significance of this work lies in its potential to support health administration, policy analysis, research, and reporting at national and district levels within Bangladesh.

7.2 Limitations

- The current database covers data from the reports collected during Sprint 1 only; not all government health documents are included.
- Scanned Portable Document Format pages could not be processed using standard text extraction methods.
- The system does not yet include a user interface or Application Programming Interface layer for non-technical users.

7.3 Future Work

- Extend coverage to additional government health reports and time periods.
- Implement Optical Character Recognition-based extraction for scanned Portable Document Format documents.
- Automate periodic data updates by monitoring government portals.

- Develop analytical dashboards for visualization of health trends.
- Integrate with the MODELBD system at the inter-sector level.
- Expose the database through a Representational State Transfer Application Programming Interface for programmatic access.

Chapter 8

Bibliography

1. Ministry of Health and Family Welfare, Bangladesh. *Health Bulletin*. Directorate General of Health Services, various years.
2. Directorate General of Health Services, Bangladesh. *Annual Report*. Ministry of Health and Family Welfare, various years.
3. Bangladesh Bureau of Statistics. *Statistical Yearbook of Bangladesh*. Ministry of Planning, Government of the People's Republic of Bangladesh.
4. Directorate General of Health Services, and National Institute of Cancer Research and Hospital, Bangladesh. *National Cancer Control Strategy and Plan of Action 2009–2015*. Ministry of Health and Family Welfare, Government of the People's Republic of Bangladesh, 2008.
5. R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*. Pearson, seventh edition.
6. A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, seventh edition.
7. W. McKinney. *Python for Data Analysis*. O'Reilly Media, third edition.
8. United Nations International Children's Emergency Fund, Bangladesh. *UNICEF Bangladesh Annual Report 2025: Delivering Results for Children Through a Difficult Year*. UNICEF, 2025.
9. Management Information System (MIS) Unit, Directorate General of Health Services, Bangladesh. *District Health Information Software 2 (DHIS2) Platform: National Routine Health Data Warehouse*. Ministry of Health and Family Welfare, Government of the People's Republic of Bangladesh.
10. Directorate General of Health Services, Bangladesh. *National Health Facility Registry Data Portal*. Management Information System (MIS) Unit, Ministry of Health and Family Welfare.
11. National Cervical and Breast Cancer Screening Programme (NCBCSP), Bangladesh. *Case-Based Cervical Cancer Screening Registry (DHIS2 Tracker Electronic Repository)*. Directorate General of Health Services, Ministry of Health and Family Welfare.